

Informe del Trabajo Integrador de AyED

Integrantes: Aguirre Joaquin Mateo, Godoy Alejandro Damian y Martínez Lautaro Nicolás.

Comisión: 5

Introducción

En esta etapa del trabajo practico se desarrollaron e implementaron distintos métodos de búsqueda y procesamiento de datos con el objetivo de comparar su funcionamiento y rendimiento.

El enfoque principal estuvo puesto en la utilización de estructuras de datos eficientes para optimizar las búsquedas dentro de una colección de información.

Explicacion del método BuscarConOtro

Primero se utiliza el método ContarOcurrencias, cuya función es recorrer todos los datos recibidos en la lista y contar cuantas veces aparece cada palabra.

Los datos llegan como una List<string>, por lo que fue necesario transformarlos en objetos de tipo Dato.

El método recorre toda la lista y verifica si el dato ya existe:

- Si ya existe, incrementa el valor de ocurrencia
- Si no existe, crea un nuevo objeto ConOcurrencia inicial igual a 1

De esta manera queda una lista donde cada palabra aparece una sola vez junto con la cantidad de veces que fue encontrada.

Luego se implementó el método BuscarConOtro, utilizando el algoritmo de ordenamiento Selection Sort.

El primer for recorre toda la lista y, en cada posición, busca cual es el elemento con mayor cantidad de ocurrencias comparando uno por uno mediante el segundo for.

Cuando encuentra el elemento con mayor ocurrencia, guarda su posición en la variable Max.

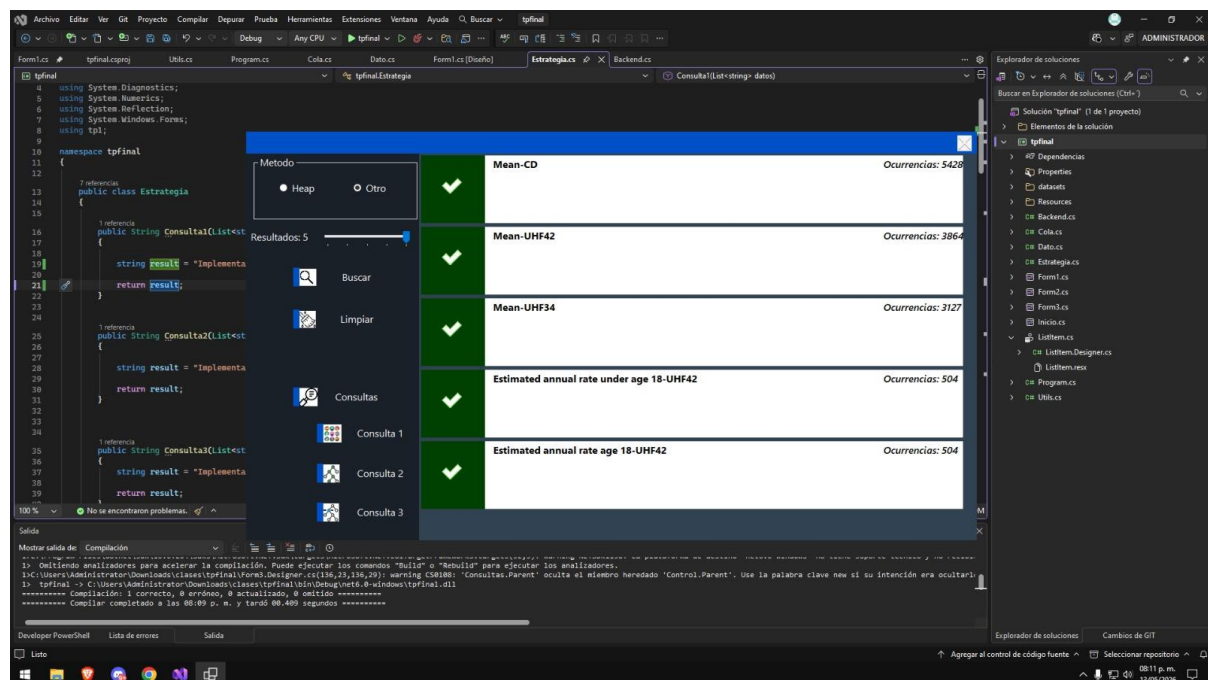
Al finalizar la búsqueda, se realiza un intercambio utilizando una variable auxiliar (aux):

El elemento con mayor ocurrencia pasa a la posición actual y el elemento que estaba anteriormente en esa posición pasa a la posición del máximo encontrado.

De esta forma la lista queda ordenada de mayor a menor según la cantidad de ocurrencias.

Finalmente, se agregan a collected únicamente los primeros elementos pedidos por parámetro mediante la variable cantidad.

Visualización del método BuscarConOtro



Explicación del método BuscarConHeap

Para implementar la búsqueda utilizando Heap, primero se utilizó el método ContarOcurrencias, ya que era necesario obtener una lista de objetos Dato con su cantidad de apariciones.

Después se implementó una estructura Max heap, donde el elemento con mayor cantidad de ocurrencias siempre queda ubicado en la raíz.

Para mantener correctamente la estructura se crearon los métodos auxiliares Heapify y ConstruirHeap.

Método Heapify

Heapify es el encargado de mantener el orden dentro del heap.

El método revisa un nodo y compara su ocurrencia con la de sus hijos izquierdo y derecho.

Si alguno de los hijos posee una ocurrencia mayor:

Primero se intercambian las posiciones pasando el mayor arriba y se vuelve a ejecutar Heapify recursivamente sobre la nueva posición.

Esto permite conservar la propiedad de Max Heap, donde el valor más grande siempre permanece arriba.

Método ConstruirHeap

El método ConstruirHeap recorre la lista desde la mitad hacia el inicio y aplica Heapify en cada posición.

El objetivo es transformar la lista completa en una estructura Heap valida.

Funcionamiento de BuscarConHeap

- El método primero cuenta las ocurrencias de cada dato.
- Una vez armado el Heap.
- El elemento con mayor ocurrencia queda en la posición 0.
- Ese elemento se agrega a collected.
- La raíz se reemplaza por el ultimo elemento del Heap
- El tamaño lógico disminuye.
- Se vuelve a ejecutar Heapify para reordenar la estructura

Este proceso se repite hasta obtener la cantidad de elementos solicitados.

Visualización del método BuscarConHeap

The screenshot displays the Visual Studio IDE with the 'BuscarConHeap' method implemented in the 'HeapifyList' class. The method is shown in the 'Form1.cs' file, and the results of the search are displayed in the 'BuscarConHeap' window.

Method Implementation (Form1.cs):

```
158     mayor = izquierda;
159 }
160
161 if (derecha < n && Lista[derecha] > Lista[mayor])
162 {
163     mayor = derecha;
164 }
165
166 if (mayor != izquierda)
167 {
168     Data aux = Lista[mayor];
169     Lista[mayor] = Lista[izquierda];
170     Lista[izquierda] = aux;
171     Heapify(Lista, n, mayor);
172 }
173
174 }
175
176 private void ConstruirHeap(List<Data> Lista)
177 {
178     for (int i = Lista.Count / 2 - 1; i >= 0; i--)
179     {
180         Heapify(Lista, Lista.Count, i);
181     }
182 }
183
184 }
185
```

BuscarConHeap Results:

Metodo	Ocurrencias
Mean-CD	Ocurrencias: 5428
Mean-UHF42	Ocurrencias: 3864
Mean-UHF34	Ocurrencias: 3127
Estimated annual rate age 18-UHF42	Ocurrencias: 504
Estimated annual rate under age 18-UHF42	Ocurrencias: 504

Output Console:

```
1> Desactivando analizadores para acelerar la compilación. Puede ejecutar los comandos "Build" o "Rebuild" para ejecutar los analizadores.
1> C:\Users\Administrator\Downloads\clases\tpfinal\Visual\Designer-cs(18,23,18,29): warning CS8881: 'Consultas.Parent' oculta al miembro heredado 'Control.Parent'. Use la palabra clave new si su intención era ocultar.
1> tpfinal -> C:\Users\Administrator\Downloads\clases\tpfinal\bin\Debug\net6.0-windows\tpfinal.dll
***** Compilación: 1 correcto, 0 erróneo, 0 actualizado, 0 omitido *****
***** Compilar completado a las 07:05 p. m. y tardó 02.467 segundos *****
```